

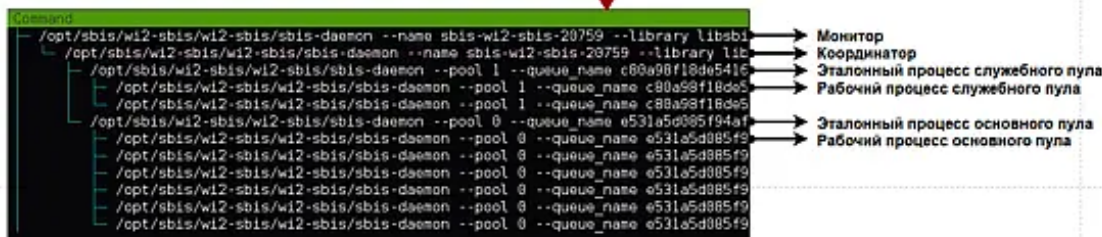
Архитектура типового узла web-сервиса

Типовой узел web-сервиса состоит из следующих процессов:

- **монитор** – корневой процесс,
- **координатор** – потомок монитора,
- процессы, соответствующие основному и служебному пулу:
 - **эталонные процессы**,
 - **рабочие процессы**.

```
1 [[ ]] 1.3% Tasks: 95; 1 running
2 [[ ]] 0.7% Load average: 0.11 0.32 0.23
3 [[ ]] 2.7% Uptime: 459 days(!), 15:21:44
4 [[ ]] 0.7%
Mem[ ] 2.38G/5.66G
Swap[ ] 0K/1.60G

PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
23781 sbis_user 20 0 10976 648 424 S 0.0 0.0 0:13.31 /opt/sbis/wi2-sbis/wi2-sbis/sbis-daemon --name sbis-wi2-sbis-20759 --library libsb1
23782 sbis_user 20 0 1124M 267M 169M S 0.0 4.6 28:22.83 /opt/sbis/wi2-sbis/wi2-sbis/sbis-daemon --name sbis-wi2-sbis-20759 --library lib
19839 sbis_user 20 0 482M 26044 6520 S 0.0 0.4 0:00.62 /opt/sbis/wi2-sbis/wi2-sbis/sbis-daemon --pool 1 --queue_name c80a98f18de5416
19851 sbis_user 20 0 689M 45536 6404 S 0.0 0.6 1:05.11 /opt/sbis/wi2-sbis/wi2-sbis/sbis-daemon --pool 1 --queue_name c80a98f18de5
19848 sbis_user 20 0 804M 181M 14348 S 0.7 3.1 16:52.53 /opt/sbis/wi2-sbis/wi2-sbis/sbis-daemon --pool 1 --queue_name c80a98f18de5
18925 sbis_user 20 0 482M 26052 6520 S 0.0 0.4 0:00.70 /opt/sbis/wi2-sbis/wi2-sbis/sbis-daemon --pool 0 --queue_name e531a5d085f94af
19803 sbis_user 20 0 710M 55700 13836 S 0.7 0.9 1:08.16 /opt/sbis/wi2-sbis/wi2-sbis/sbis-daemon --pool 0 --queue_name e531a5d085f9
19800 sbis_user 20 0 713M 56224 14828 S 0.6 1.0 1:07.69 /opt/sbis/wi2-sbis/wi2-sbis/sbis-daemon --pool 0 --queue_name e531a5d085f9
18997 sbis_user 20 0 710M 56612 16312 S 0.0 1.0 1:10.30 /opt/sbis/wi2-sbis/wi2-sbis/sbis-daemon --pool 0 --queue_name e531a5d085f9
18942 sbis_user 20 0 710M 65432 21816 S 0.7 1.1 1:12.87 /opt/sbis/wi2-sbis/wi2-sbis/sbis-daemon --pool 0 --queue_name e531a5d085f9
18940 sbis_user 20 0 725M 73700 23660 S 0.0 1.2 1:16.43 /opt/sbis/wi2-sbis/wi2-sbis/sbis-daemon --pool 0 --queue_name e531a5d085f9
18937 sbis_user 20 0 733M 75480 24196 S 0.0 1.3 1:22.21 /opt/sbis/wi2-sbis/wi2-sbis/sbis-daemon --pool 0 --queue_name e531a5d085f9
```



Структура процессов сервиса

Монитор – первый процесс web-сервиса, который запускается. Задача монитора: вызвать `fork()`, запустить дочерний процесс (координатор) и ждать его завершения, либо перезапустить координатор, если он завершен аварийно.

Координатор – основной процесс web-сервиса. Он принимает запросы, рассылает ответы и занимается балансировкой нагрузки. Когда мы обращаемся на сервис, запрос приходит в координатор. В координаторе находятся сущности, предназначенные для приема запросов – [точки входа](#).

В типовой конфигурации координатор сам запросы не исполняет – этим занимаются **рабочие процессы**, которые делятся на пулы:

- **основной** (в мониторе процессов отображается как pool 0) – предназначен для исполнения более приоритетных запросов; например запросы с БЛ, запросы с интерфейса.
- **служебный** (в мониторе процессов отображается как pool 1) – предназначен для исполнения "тяжеловесных" запросов, зачастую асинхронных.

У каждого пула Linux есть один **эталонный процесс** и, собственно, **рабочие процессы**. На Windows эталонных процессов нет.

Эталонный процесс предназначен для одной цели – быстро воспроизводить **рабочие процессы**. Эталонный процесс во время старта подгружает все, что можно сделать заранее и "зависает" в этой точке. Дальше он принимает только запросы на `fork()` самого себя. Такое поведение позволяет в случае сбоя одного из рабочих процессов очень быстро запустить новый рабочий процесс.

Как это происходит: в результате `fork'a` эталонного процесса запускается почти готовый рабочий процесс, который, после дополнительной настройки, готов к использованию. Сам эталонный процесс ничего не делает и потребляет незначительный объем памяти.

Рабочий процесс занимается обработкой запросов. Координатор принимает запрос и передает рабочим процессам на исполнение. Рабочие процессы выполняют запрос, передают ответ в разделяемую память и уведомляют координатор о том, что можно забирать ответ.

Рабочий процесс в типовой конфигурации исполняет запросы в один поток. Однако, с помощью параметров [Ядро.Сервер приложений.ЧислоПотоковВРабочихПроцессах](#) и [Ядро.Сервер приложений.ЧислоПотоковВСлужбныхРабочихПроцессах](#), он может быть перенастроен для работы в многопоточном режиме.

Работа в многопоточном режиме позволяет более оптимально использовать ресурсы операционной системы, однако сопряжена с некоторыми рисками, т.е. для успешного выполнения работы необходимо, чтобы код методов бизнес-логики был потокобезопасным.

При разработке на Python и JavaScript также следует помнить, что интерпретаторы данных языков программирования работают однопоточно, соответственно, если методы бизнес-логики содержат существенную вычислительную часть – разпараллелить их потоками не получится (это можно сделать только процессом).

Структура организации файлов web-сервиса

Корневой каталог сервиса содержит следующие файлы:

1. Файлы, обеспечивающие функциональность ядра платформы, бинарные компоненты ядра платформы и их зависимости. Это файлы с расширением `.so`.
2. Конфигурационный файл `sbis-rpc-service.ini`.

```
1 [Базовая конфигурация]
2 Конфигурация=Адрес=http://test-cloud-ctrl-private.sbis.ru/cfg
```

3. Файл `version.xml` – в нем прописаны версия ядра платформы и сервиса (с номером и датой сборки).

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <version>
3
4   <version_core>
5     <version_number>24.2100</version_number>
6     <build_number>270</build_number>
7     <build_date>24.05.2024</build_date>
8     <build_time>16:03:59</build_time>
9   </version_core>
10
11   <version_number>24.2153</version_number>
12
13   <build_number>12</build_number>
14
15   <build_date>29.05.2024</build_date>
16
17   <build_time>06:16:05</build_time>
18
19 </version>
```

Также корневой каталог может содержать папки:

- **modules** – для модулей, используемых в реализации функционала сервиса,
- **logs** – для логов,
- **asynccall** – для асинхронных вызовов, сохраненных на данном узле БЛ,
- **resources** – для статических ресурсов.

Последовательность процесса запуска web-сервиса

Для запуска веб-сервиса используется универсальный запускаемый файл `sbis-daemon`, которому в [systemd](#) прописана команда `start` с подгрузкой библиотеки `sbis-rpc-service300.so`.

1. **sbis-daemon** при старте запускает [fork\(\)](#) – функцию, которая создает копию его процесса – первый процесс веб-сервиса, монитор.

2. Монитор выполняет [fork\(\)](#) самого себя, тем самым запуская дочерний процесс (координатор), и "зависает", ожидая его завершения.
3. Координатор загружает библиотеку **sbis-rpc-service300.so**, с которой загружается все ядро.
4. Управление переходит к библиотеке **sbis-loader.so**. Библиотека проводит поиск бинарных файлов в каталоге с модулями. Бинарные файлы загружаются в адресное пространство процесса. Результат этого шага – процесс, в адресное пространство которого подгружены ядро и бинарные файлы модулей.
5. Ядро обращается к **фабрике конфигураций** – она предоставляет провайдер конфигурационных данных для сервиса. Здесь следует отметить, что для сервиса возможен один из двух видов получения конфигурации:
 - конфигурация по умолчанию – вся конфигурация прописана в файле **sbis-rpc-service.ini**;
 - удаленный источник конфигурации – конфигурация предоставляется модулем Cloud Interaction; этот источник позволяет запрашивать конфигурацию данного узла web-сервиса в сервисе ["Управление облаком"](#).
6. Координатор получает конфигурацию и начинает вызывать функцию инициализации модулей, которая регистрируется каждым модулем при загрузке его бинарного файла в адресное пространство.
7. В процессе инициализации модулей координатор строит дерево зависимостей, по информации из <depends> .s3mod-файлов описания модуля. Исходя из построенного дерева зависимостей формируется порядок загрузки модулей.
8. Координатор вызывает функцию OnCleanup(), которая отвечает за освобождение ресурсов, задействованных во время загрузки модуля. Функция вызывается для каждого модуля, в обратном порядке – от самого последнего до базовых модулей.
9. Через исполняемый файл sbis-daemon запускаются эталонные процессы с определенными аргументами. Такими аргументами могут быть:
 - тип пула (основной или служебный)
 - реквизиты разделяемой памяти, через которую будет идти обмен с координатором (имена сегментов, число потоков).

Сначала запускаются эталонный, затем рабочие процессы основного пула, потом эталонный и рабочие процессы служебного пула.
10. Эталонный процесс получает конфигурацию от координатора, загружает все модули в адресное пространство процесса, вызывает функцию OnEndLoadKernelModules(), оповещает координатора о загрузке и ожидает указаний на [fork\(\)](#).
11. Координатор последовательно посылает команды на [fork\(\)](#) эталонному процессу.
12. Эталонный процесс запускает рабочий процесс для соответствующего пула и вызывает функции OnEndLoadModules(), OnCleanup().
13. Рабочий процесс готов к работе, о чем оповещает координатора.
14. После того, как все пулы запущены, координатор открывает точки входа.